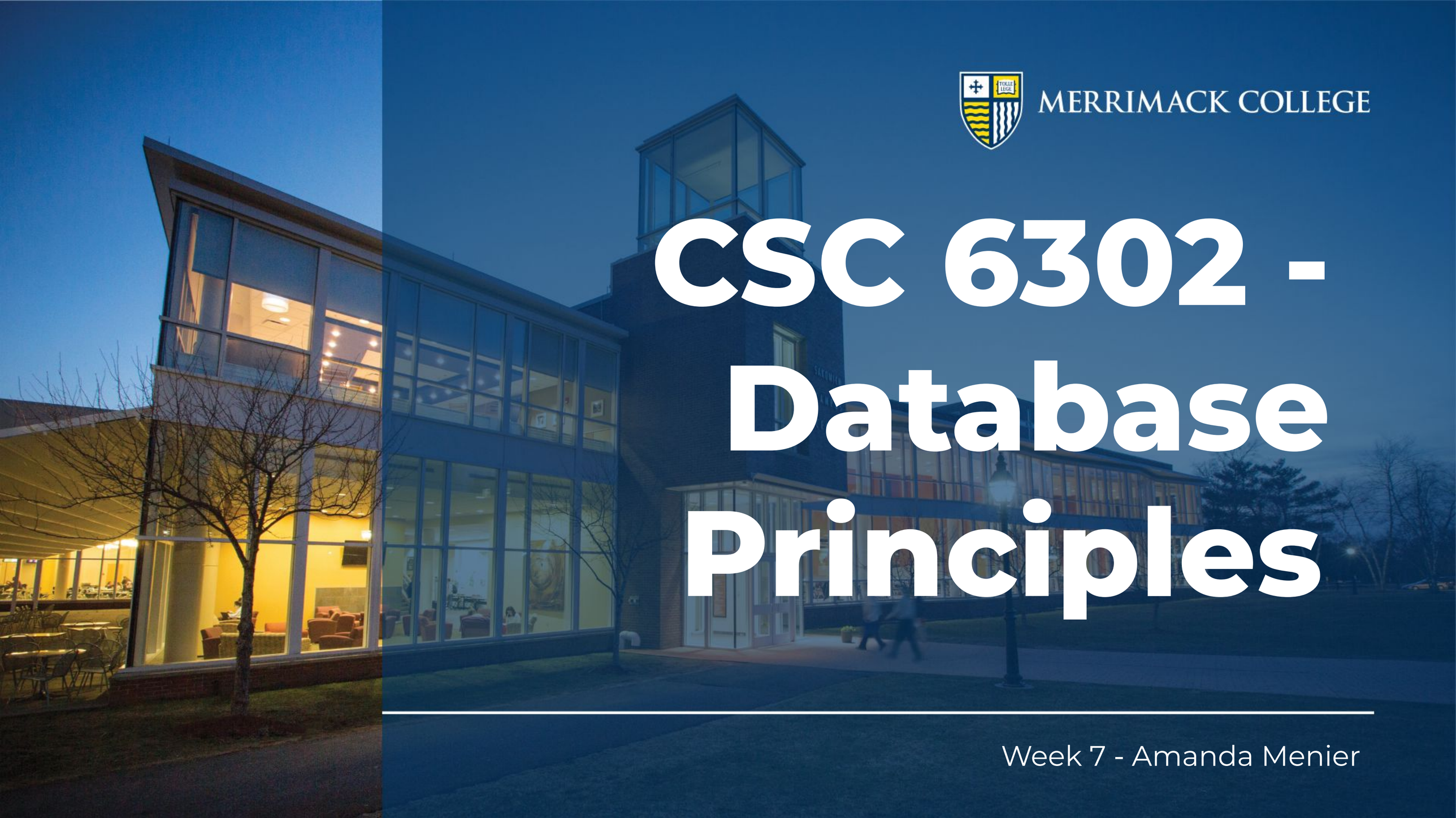


MERRIMACK COLLEGE

CSC 6302 - Database Principles

Week 7 - Amanda Menier

Agenda

- Fixed and Variable Length Storage
- Heap File Organization
- Sequential File Organization
- Clustering & Non-clustering Indexes
- Buffer and Manager
- B+ Trees
- Creating Index in SQL

The database is stored as a collection of *files (Table)*

Each file is a sequence of *records (row)*

A record is a sequence of fields (columns)

Record is mapped to disk blocks

Fixed-Length Records

Fixed-Length Records

Assumptions

Record size is fixed

Each file has a single record type only

Different files are used for different relations

Fixed-Length Records

Simple approach:

Store record *i* starting from byte *n* * (*i* – 1), where *n* is the size of each record.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

Deletion of record *i:*

move records *i* + 1 to i, …

Record 3 deleted

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

Deletion of record *i:* alternatives*:*

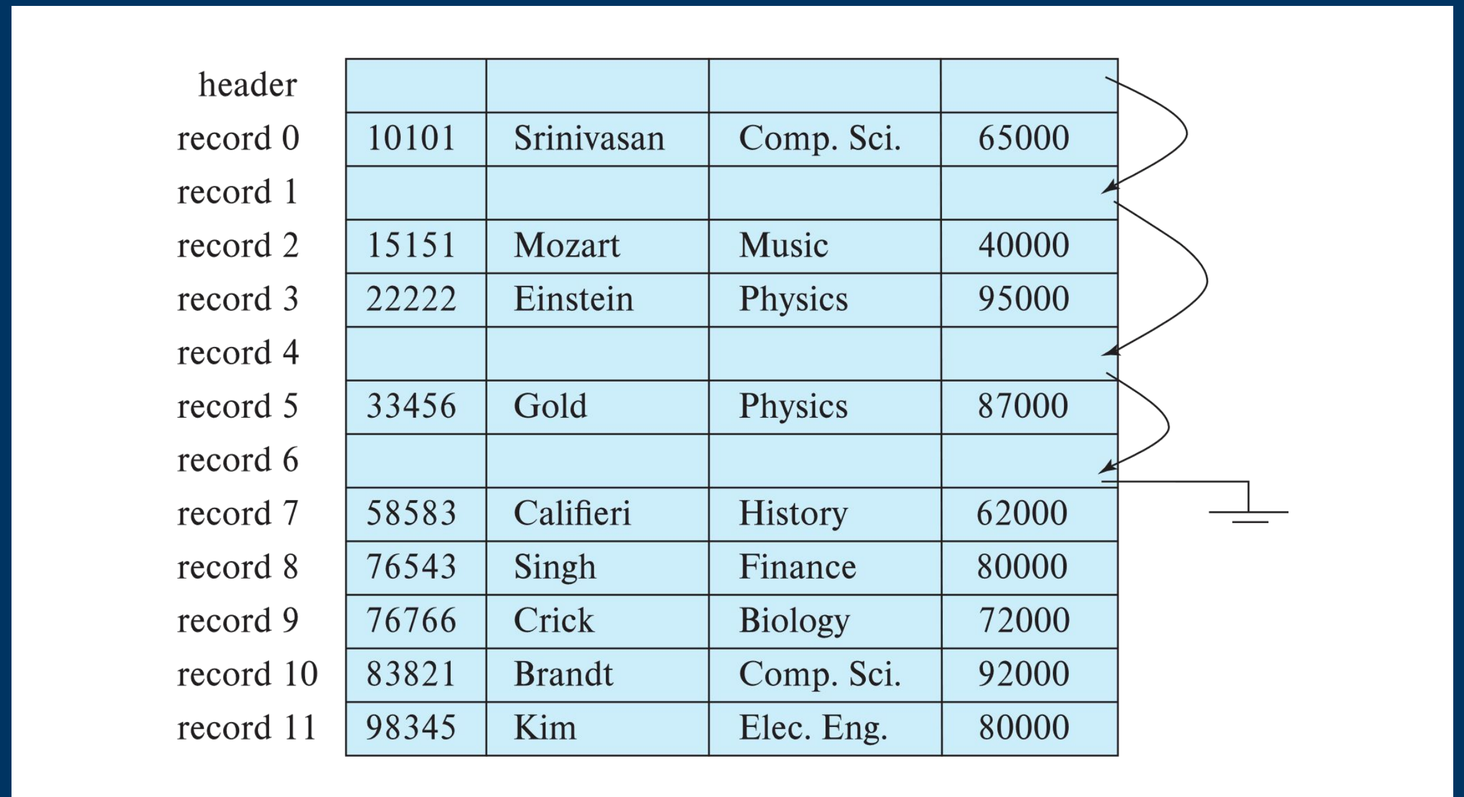
- **move record *n* to *i***
- **Record 3 deleted and replaced by record 11**

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Fixed-Length Records

Deletion of record *i:*

do not move records, but link all free records on a *free list*



Variable-Length Records

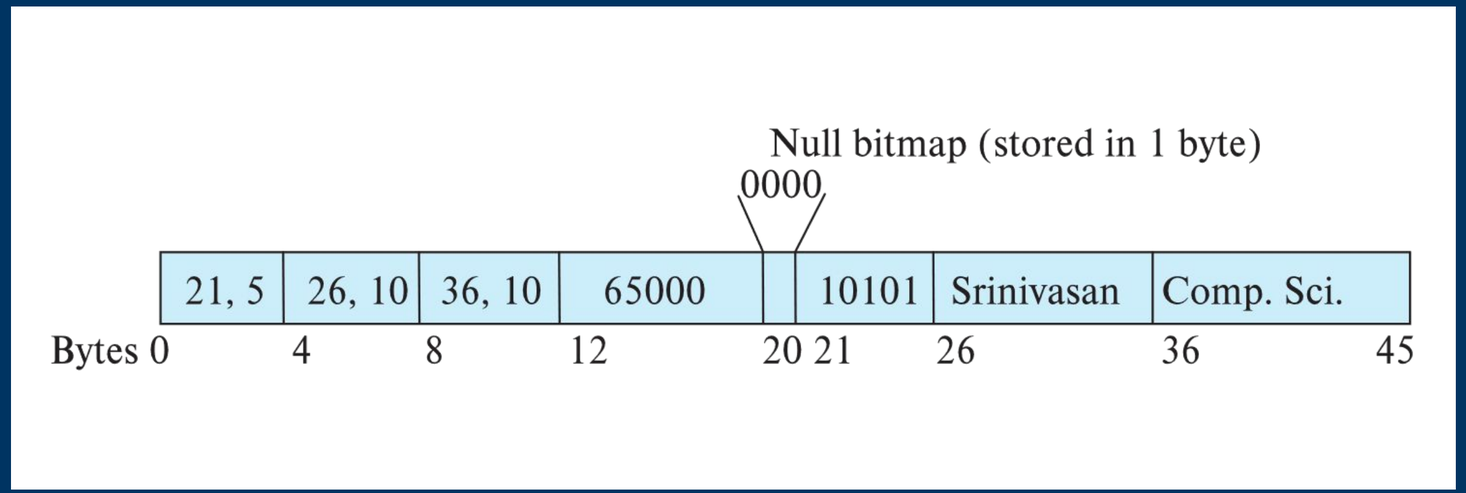
Variable-length records arise in database systems in several ways:

- Storage of multiple record types in a file.
- Record types that allow variable lengths for one or more fields such as strings (**varchar**)

Attributes are stored in order

Variable length attributes represented by fixed size (offset, length), with actual data stored after all fixed length attributes

Null values represented by null-value bitmap



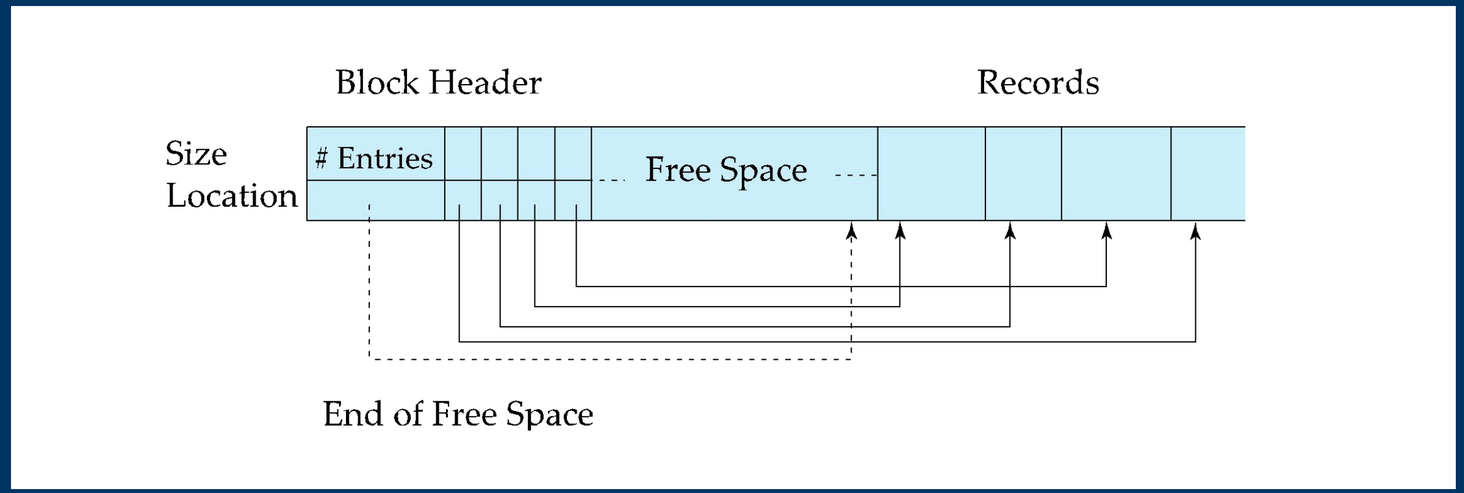
Variable-Length Records

Slotted page header contains:

- number of record entries
- end of free space in the block
- location and size of each record

Records can be moved around within a page to keep them contiguous with no empty space between them; entry in the header must be updated.

Pointers should not point directly to record — instead they should point to the entry for the record in header.



Organization of Records in Files

Heap – record can be placed anywhere in the file where there is space

Sequential – store records in sequential order, based on the value of the search key of each record

In a **multi-table clustering file organization** records of several different relations can be stored in the same file

Motivation: store related records on the same block to minimize I/O

B^+-tree file organization

Ordered storage even with inserts/deletes

Hashing – a hash function computed on search key; the result specifies in which block of the file the record should be placed

Heap File Organization

Records can be placed anywhere in the file where there is free space

Records usually do not move once allocated

Important to be able to efficiently find free space within file

Free-space map

Array with 1 entry per block. Each entry is a few bits to a byte, and records fraction of block that is free

In example below, 3 bits per block, value divided by 8 indicates fraction of block that is free

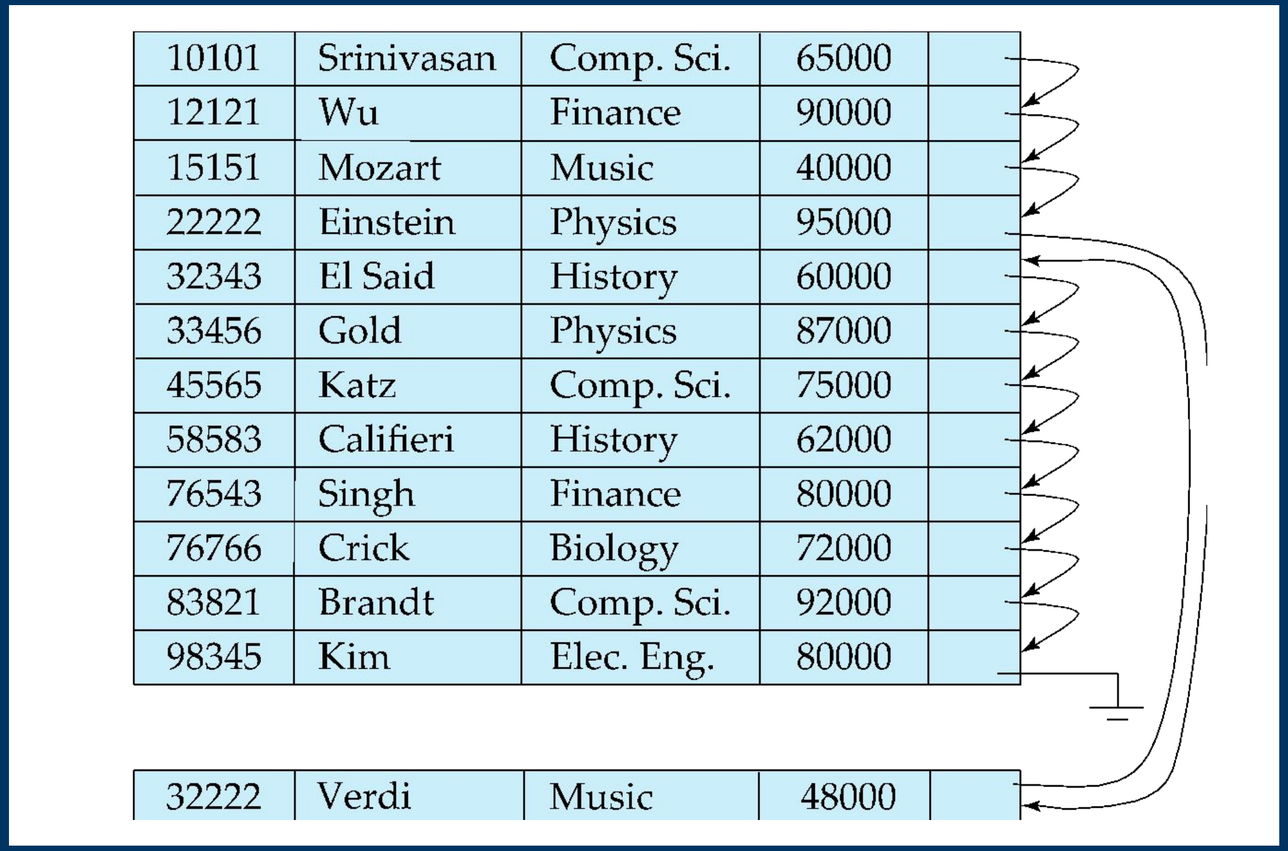
Can have second-level free-space map

In example below, each entry stores maximum from 4 entries of first-level free-space map

Sequential File Organization

Suitable for applications that require sequential processing of the entire file

The records in the file are ordered by a search-key



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

32222	Verdi	Music	48000	

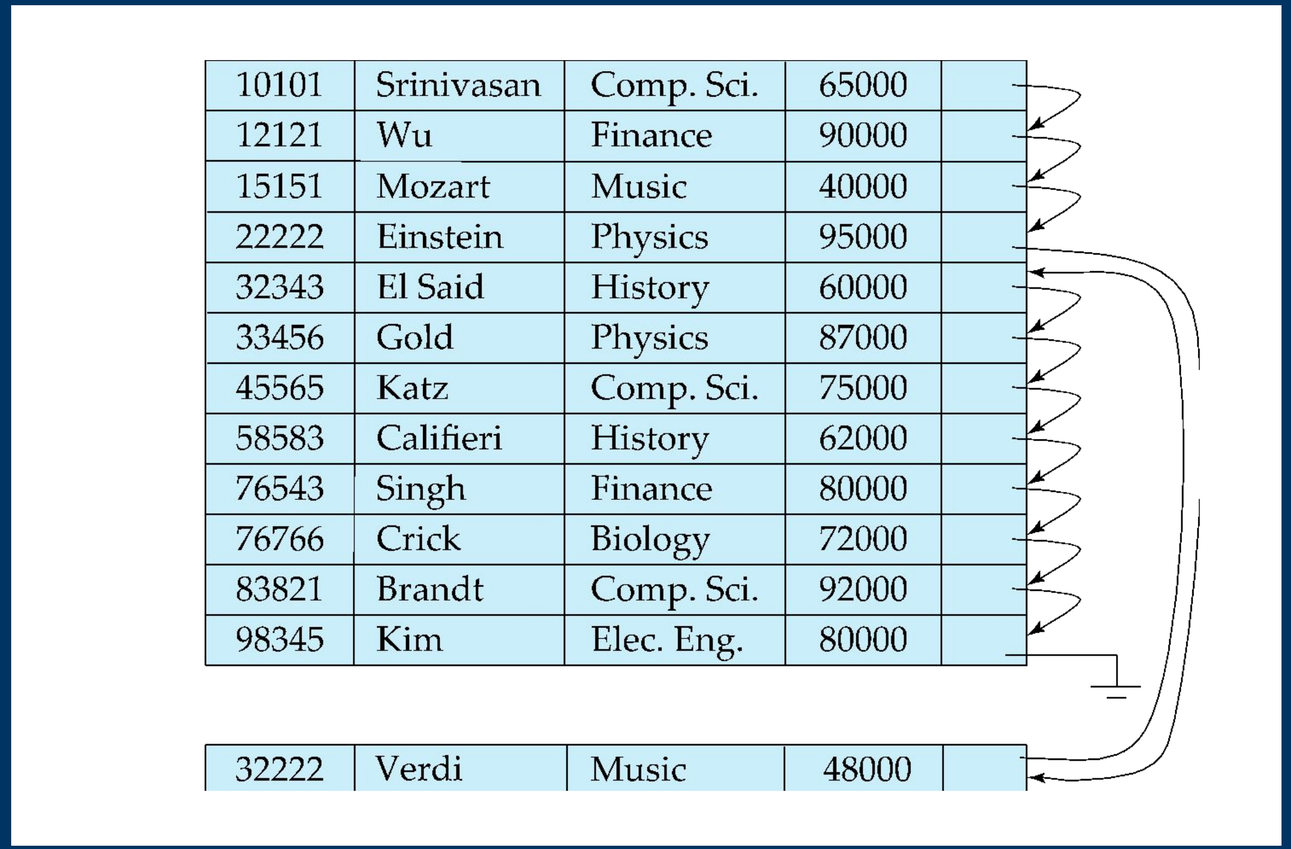
Sequential File Organization

Deletion – use pointer chains
Insertion –locate the position where the record is to be inserted

- if there is free space insert there
- if no free space, insert the record in an overflow block
- In either case, pointer chain must be updated

Need to reorganize the file

- from time to time to restore
- sequential order



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

32222	Verdi	Music	48000	

Multi-table Clustering File Organization

Store several relations in one file using a **multitable clustering** file organization

department

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Physics	Watson	70000

instructor

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

Multi-table clustering of *department* and *instructor*

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

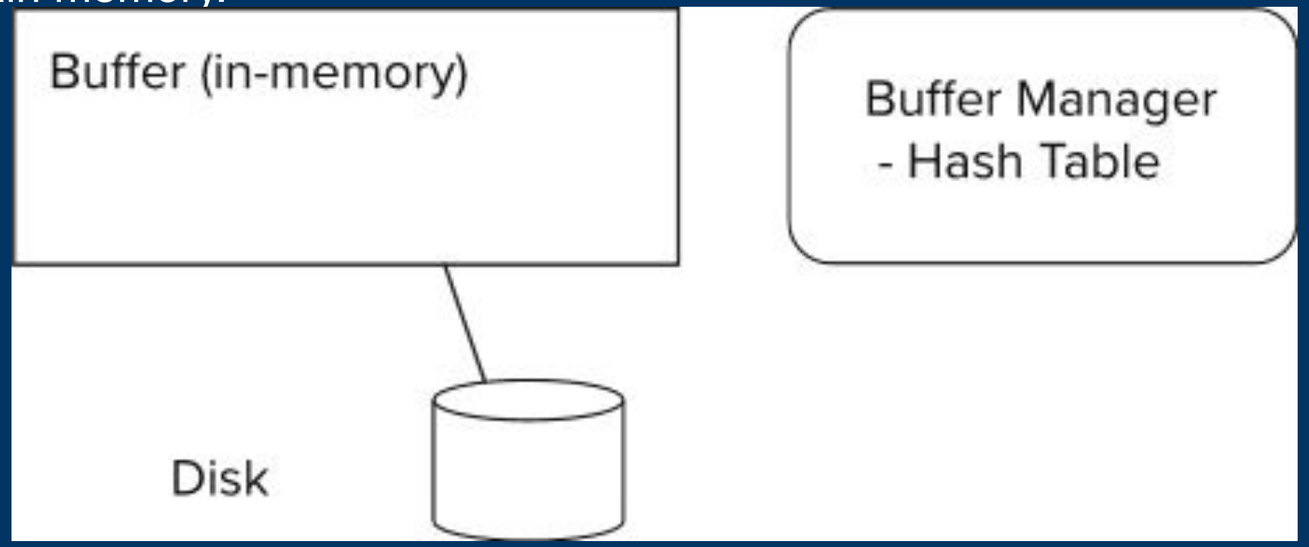
Storage Access

Blocks are units of both storage allocation and data transfer.

Database system seeks to minimize the number of block transfers between the disk and memory. We can reduce the number of disk accesses by keeping as many blocks as possible in main memory.

Buffer – portion of main memory available to store copies of disk blocks.

Buffer manager – subsystem responsible for allocating buffer space in main memory.



Buffer Manager

Programs call on the buffer manager when they need a block from disk.

- If the block is already in the buffer, buffer manager returns the address of the block in main memory
- If the block is not in the buffer, the buffer manager
  - Allocates space in the buffer for the block
    - Replacing (throwing out) some other block, if required, to make space for the new block.
    - Replaced block written back to disk only if it was modified since the most recent time that it was written to/fetched from the disk.
  - Reads the block from the disk to the buffer, and returns the address of the block in main memory to requester.

Indexes and Storage

Databases make use of the storage topics we have discussed
- Organizing storage of data in main memory
- Buffering in volatile memory speeds up performance

An index is a tool used by database developers to give the database engine hints on how it should store the data
An index allows faster lookup in volatile memory, if used properly

All primary key and foreign key columns have an index by default

Which makes sense, as we use these columns a lot when we query!

As an index has storage overhead, it's important to only add an index when its usefulness outweighs its storage needs

Basic Concepts For Indexing

Indexing mechanisms used to speed up access to desired data.

E.g., author catalog in library

Search Key - attribute to set of attributes used to look up records in a file.

An **index file** consists of records (called **index entries**) of the form

Index files are typically much smaller than the original file

Two basic kinds of indices:

Ordered indices: search keys are stored in sorted order

Hash indices: search keys are distributed uniformly across "buckets" using a "hash function".

Ordered Indexes

In an **ordered index,** index entries are stored sorted on the search key value.

Clustering index: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.

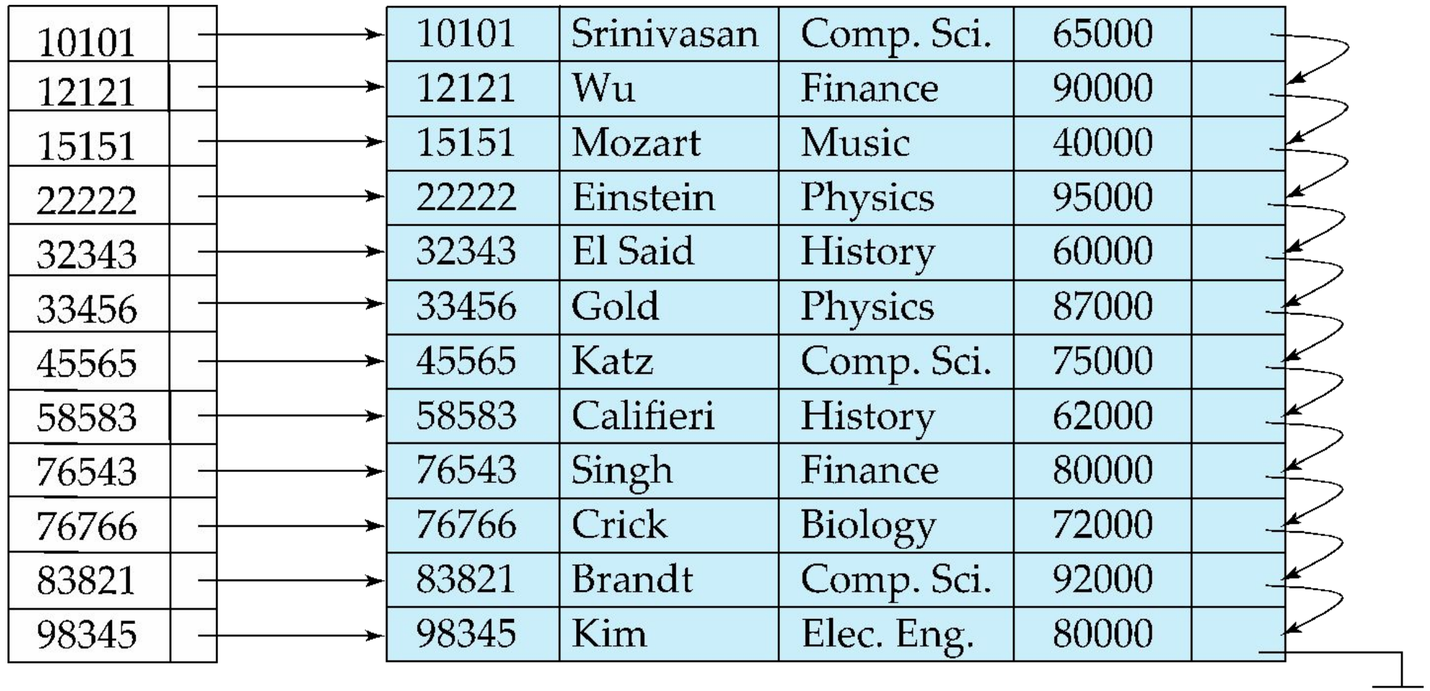
- Also called **primary index**
- The search key of a primary index is usually but not necessarily the primary key.

Secondary index: an index whose search key specifies an order different from the sequential order of the file. Also called **non-clustering index.**

Index-sequential file: sequential file ordered on a search key, with a clustering index on the search key.

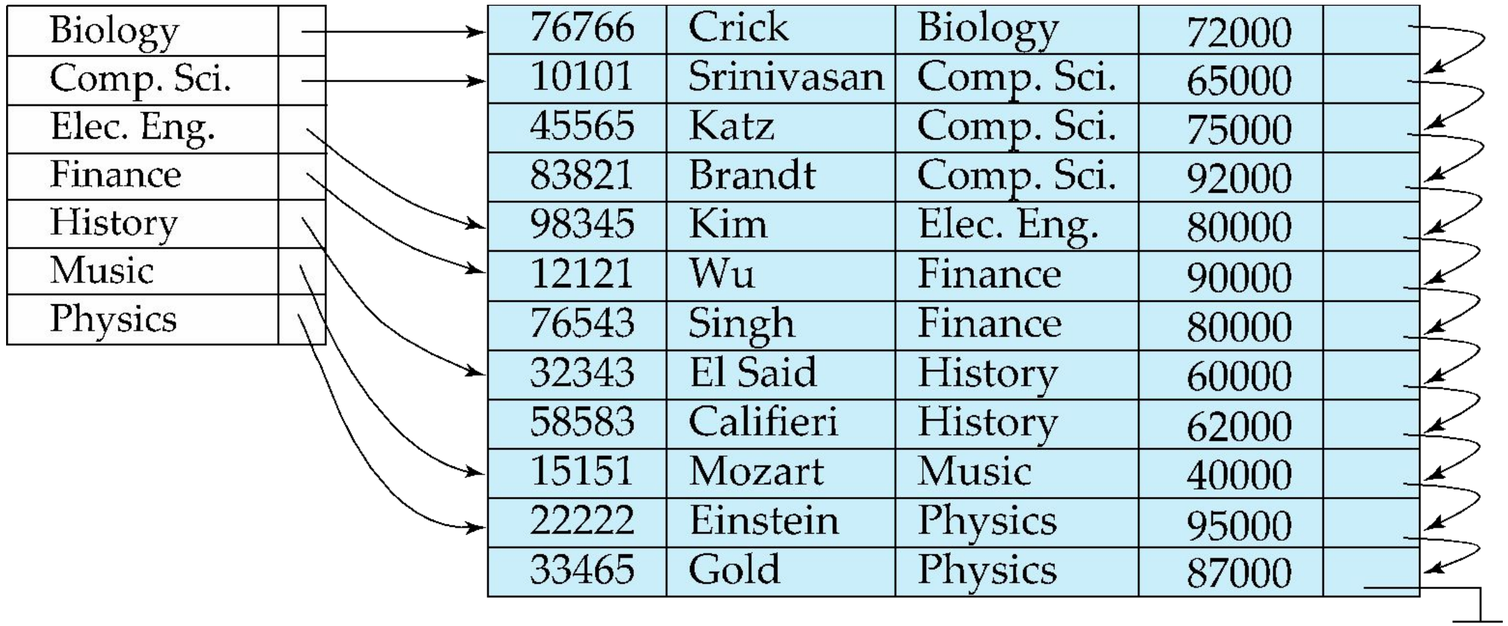
Index Files

E.g. index on *ID* attribute of *instructor* relation



Index Files (Cont.)

E.g. index on *ID* attribute of *instructor* relation



Sparse Index Files

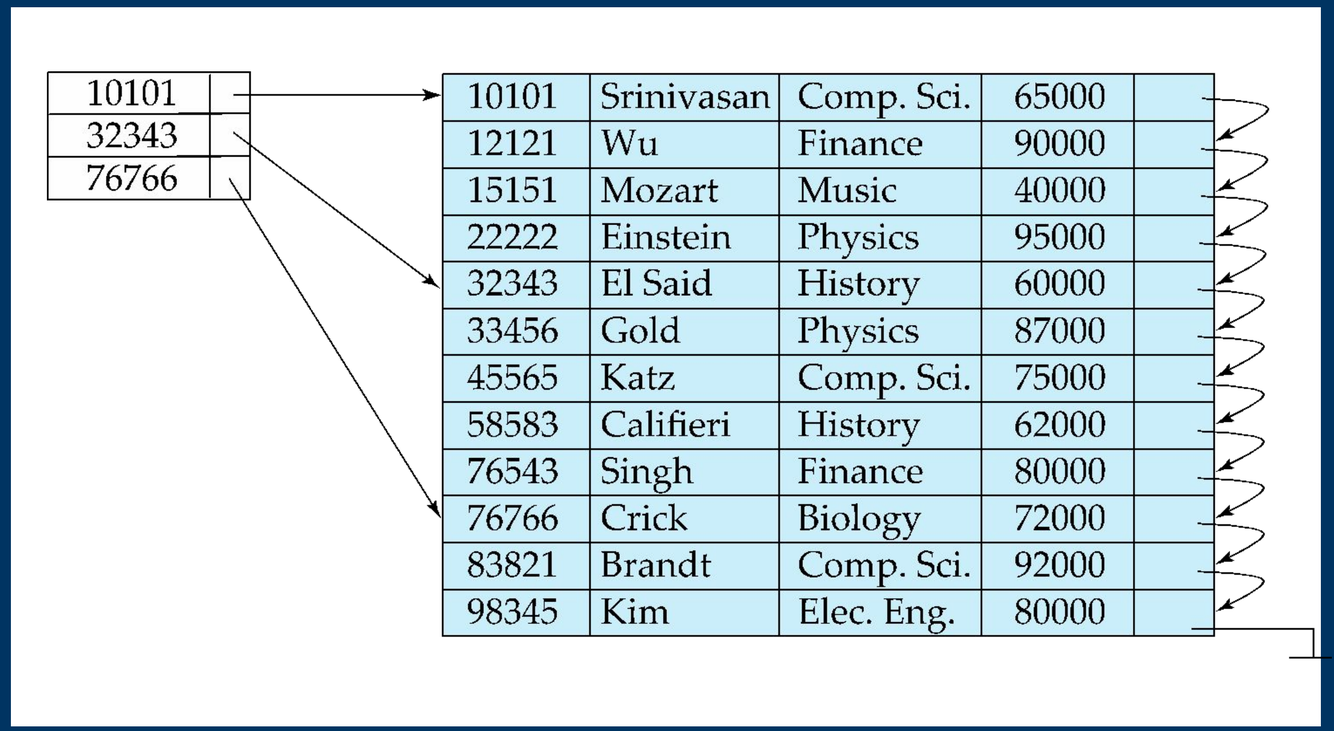
Sparse Index: contains index records for only some search-key values.

Applicable when records are sequentially ordered on search-key

To locate a record with search-key value *K* we:

Find index record with largest search-key value < *K*

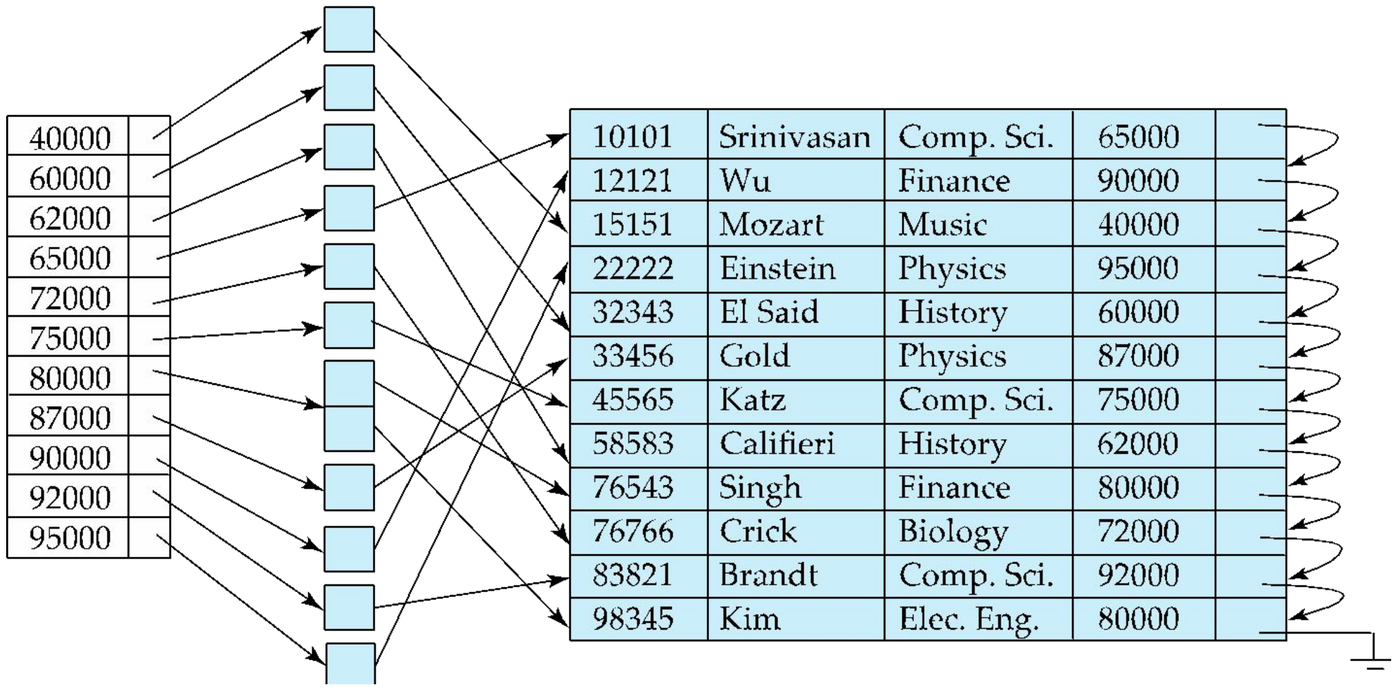
Search file sequentially starting at the record to which the index record points



Secondary Indices Example

Secondary index on salary field of instructor

Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.



Multilevel Index

If index does not fit in memory, access becomes expensive.

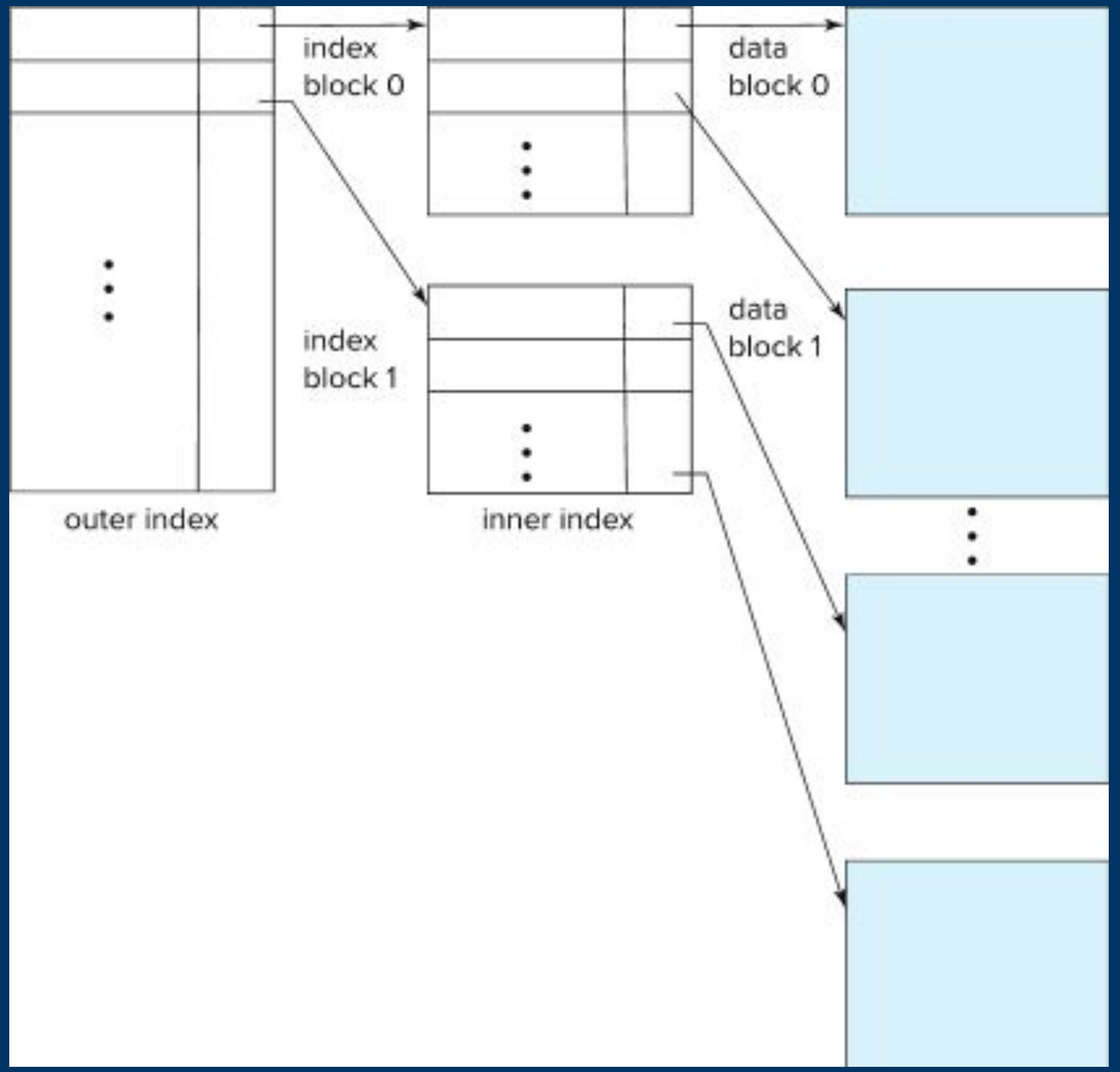
Solution: treat index kept on disk as a sequential file and construct a sparse index on it.

- outer index – a sparse index of the basic index
- inner index – the basic index file

If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.

Indices at all levels must be updated on insertion or deletion from the file.

Multilevel Index (Cont.)



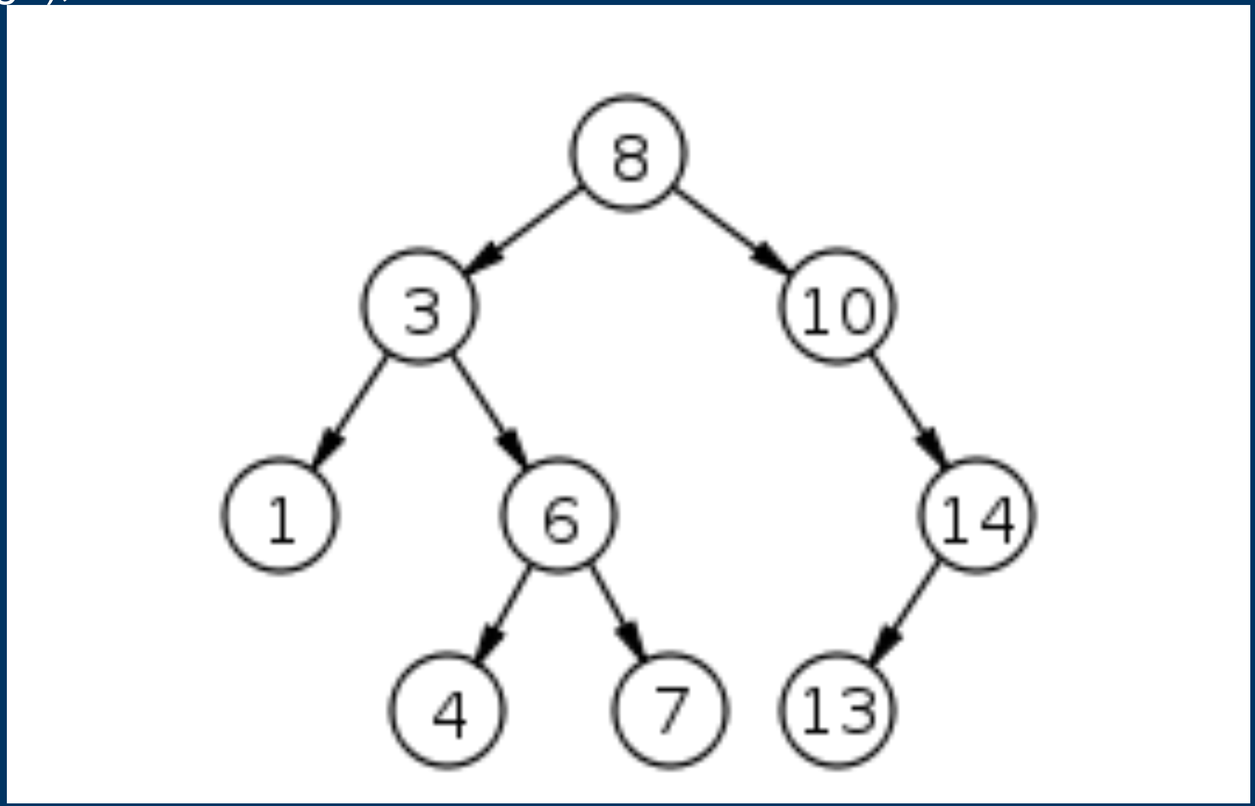
What Data Structure Works Best?

We could use a simple array of pointers, that point to the actual data in main memory.

Linear search times, O(n) are pretty slow. We can do better

What about a tree structure?

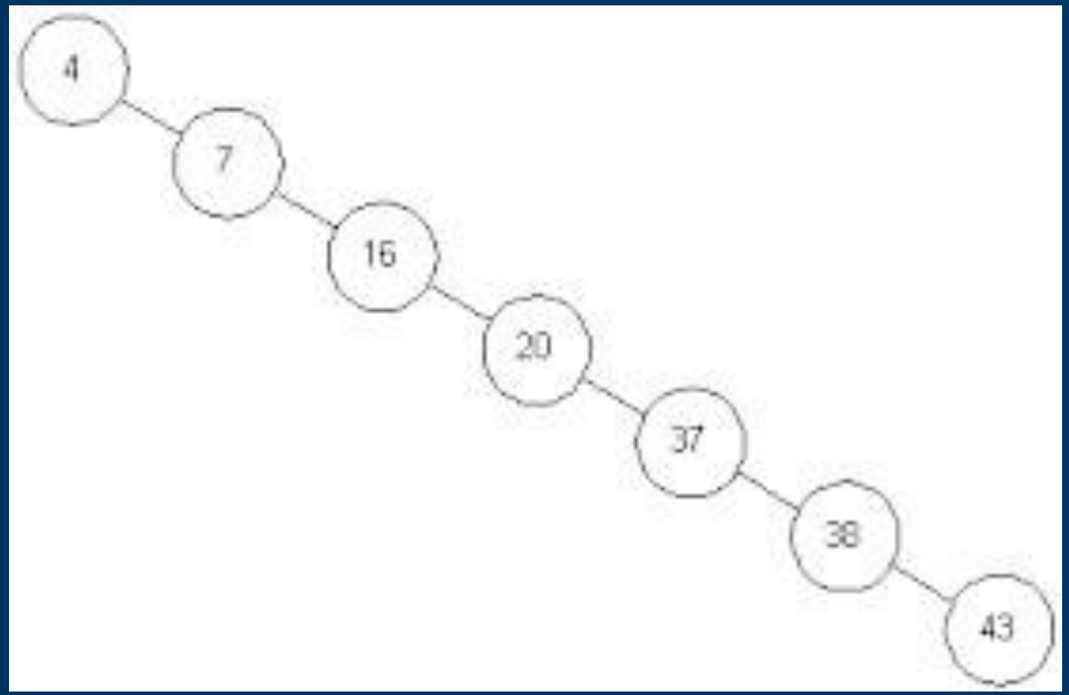
On average, a binary search tree improves search time from linear to O(logn), where n is the number of nodes in the tree



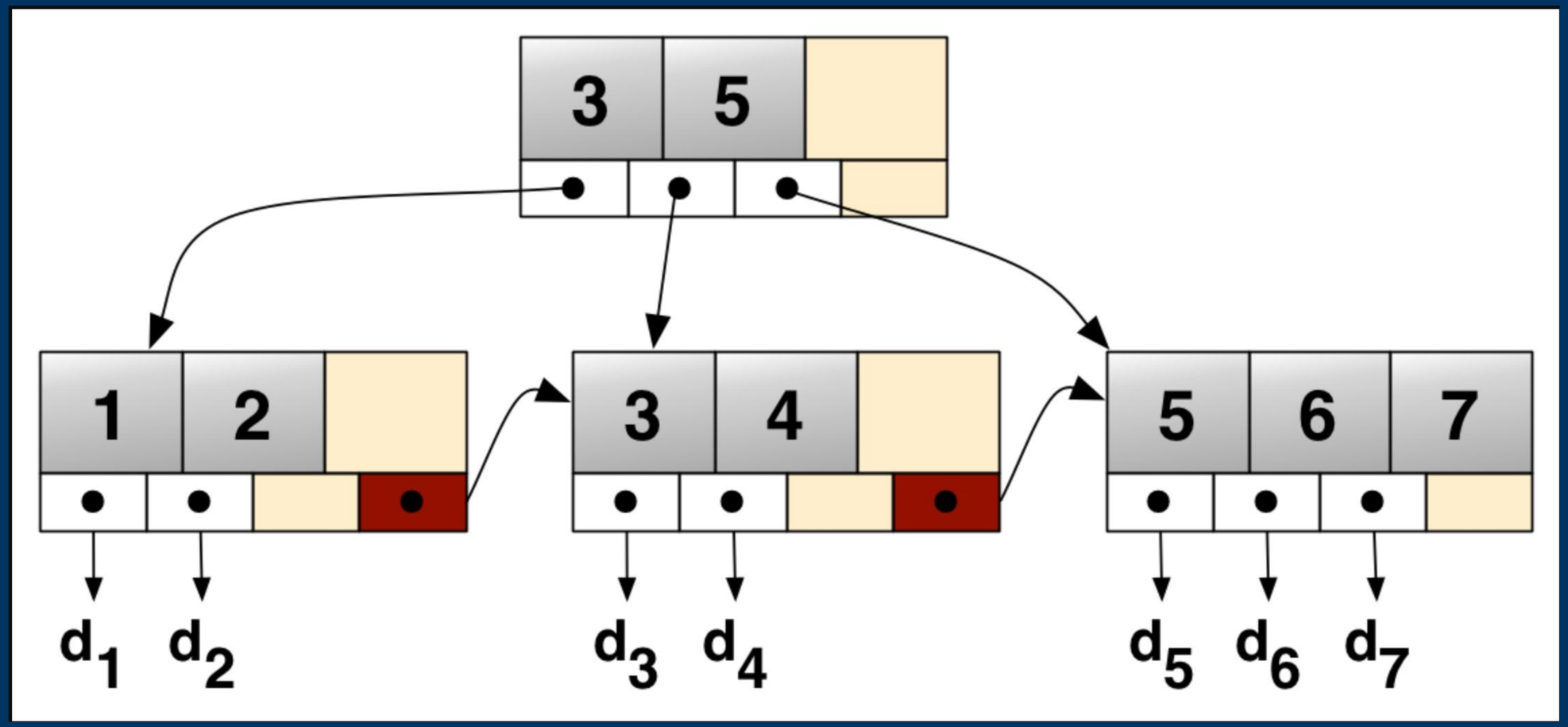
Worst Case Binary Search Tree

Here’s an obvious reason why a binary search tree isn’t good enough. Consider building a tree from inputs: 4, 7, 16, 20, 37, 38, 43

This is just a linked list….so linear time in the worst case!



Example of B^+-Tree



Creating a B+ Tree

For internal nodes:

Can have up to n pointers (children).

Contains up to n - 1 keys.

For leaf nodes:

Contains up to n - 1 keys and a pointer to the next leaf node (for sequential access)

Creating a B+ Tree

Promotion

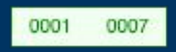
When a node (leaf or internal) becomes full after an insertion, it must be split into two nodes. The median key is promoted to the parent node to maintain balance.

If the parent node also becomes full, the process continues recursively up the tree, possibly requiring a split at the root and creating a new root

B+ Tree Example

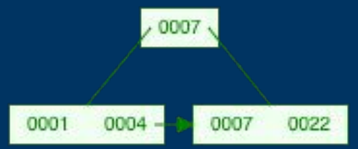
Create a B+(3) Tree with the following data: 1, 7, 22, 4

Step One:
Insert the first two values,
no problem



Step 2: Adding 22 means we need to add a temp node with 1, 7 and 22…but its too big. We need to split the node into two. Note 007 is in the leaf level and promoted up.

Step Three:
Add 4 to the leaf node, and no need to split



B+ Tree Visualization

Definitely take a look at this visualization tool:

https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html

Index Definition in SQL

Create an index

create index <index-name> **on** <relation-name>
(<attribute-list>)

E.g.,: CREATE INDEX users_name_index ON users(last_name);

Use **create unique index** to indirectly specify and enforce the condition that the search key is a candidate key.

Not really required if SQL **unique** integrity constraint is supported

To drop an index

drop index <index-name>

Most database systems allow specification of type of index, and clustering.